Lab 1 A Statistical Analysis of Vowel Inventories of World Languages

Report - Lab1

Peng CHEN M2 CL 22409126 Paris Cité Université

In this lab, I worked with PHOIBLE v2.0 to compare vowel inventories across languages. For each Glottocode, I kept the inventory with the highest InventoryID and counted every segment with SegmentClass == "vowel" as a vowel, including marginal segments. I first looked at vowel inventory sizes and outliers, then tested the two proposed linguistic universals. I also analysed phoneme associations using statistical tests, effect sizes, a null model, genealogically balanced sampling, and Apriori.

The results suggest that the first universal is better seen as a strong statistical tendency than as an absolute universal. About 79.8% of languages contain [i], [a], and [u] together, rising to about 86.4% when close equivalents are included. The second universal is also supported, but not equally strongly in every case: the nasal-to-oral implication holds for about 88.4% of relevant languages, the long-to-non-long implication for about 77.6%, and the [y]-to-[i]+[u] implication for about 89.8%.

The main difficulty was understanding some of the statistical methods and implementing them correctly. Coding was also a challenge. Because of the limited time and my recent focus, I couldn't realistically write every part manually. So I used AI mainly to help with the concrete implementation of some algorithms and more complicated code.

At the same time, I was very willing to learn from the generated code. I used the time saved to focus more on understanding the ideas behind the algorithms, designing the overall and detailed workflow, thinking through the pseudocode, and getting more familiar again with commonly used Python libraries and methods. I also made sure that I could read and check the generated code, find where a problem came from, and debug it manually. I paid attention to its structure and coding style. For me, being able to design a solution, understand and correct some AI-generated code, and write the main logic or pseudocode myself is more useful than simply trying to type every line without help.

Another important thing I learnt from this lab was to get back into the habit of thinking carefully about data analysis. It reminded me that getting a number is only one part of the work. We also have to ask whether the data have problems, whether the method is suitable, whether the result really supports the hypothesis, and if not, how far we can actually go with the conclusion. For example, a low p-value does not necessarily mean that an association is strong, especially with a

large sample, so effect size also matters. Multiple testing can create many false positives, and rare phoneme pairs may require Fisher’ s exact test instead of the chi-square test. The null model and genealogically balanced sampling also showed why the choice of baseline and sample can change how we interpret a result.

More generally, this lab helped me review and think about the logic behind some statistical and data-analysis methods: what problem they solve, when we should use them, and what their limitations are.

To read the report online: Lab 1 A Statistical Analysis of Vowel Inventories of World Languages

To read the code of this lab online : https://colab.research.google.com/drive/1Nl_wJuxmyrU4PE6NTUgMQxjCEqQFoeWs?usp=sharing

The data are distributed as a single table. Download the version tagged v2.0: https://raw.githubusercontent.com/phoible/dev/v2.0/data/phoible.csv
Lab1 instructions: https://pages.llf-paris.fr/~gwisniewski/assets/2026/intro-why-multilinguality-pdfjam.pdf

2.Getting the Data

1. **Have you read and understood the instructions?**

Yes, I've read and understood the instructions.

2. **Load the file and report three numbers: how many rows it contains, how many distinct inventories (column InventoryID), and how many distinct languages (column Glottocode). Why are these three numbers different?**

Code block

```
df = pd.read_csv(path)
print("Number of rows:", len(df))
print("Number of inventories:", df["InventoryID"].nunique())
print("Number of Glottocodes:", df["Glottocode"].nunique())
```

> Number of rows: 105467
> Number of distinct inventories: 3020
> Number of distinct languages (Glottocodes): 2184

Each row represents one phoneme in one inventory. An inventory is the set of phonemes reported for a language in a particular linguistic description or source. Therefore, one inventory

contains many phonemes and is represented by many rows. One language can also have several inventories because it may have been described by different sources or linguists. This is why these three numbers are different.

3. **Why are there several ISO standards for identifying languages (and what are they)?**

Because of the different needs and levels of coverage. For language identification, ISO developed several standards within the ISO 639 family. They are : ISO 639-1 uses two-letter codes for major languages, ISO 639-2 and ISO 639-3 use three-letter codes with broader coverage, and ISO 639-5 is used for language groups.

Why did linguists also introduce Glottocodes? Is that a clever idea?

Because ISO codes are mainly designed to identify individual languages and are not detailed enough for all linguistic entities. Glottocodes can uniquely identify languages, dialects/varieties and language families.

It is a clever idea. They make linguistic databases easier to link and compare consistently.

https://www.iso.org/iso-639-language-code?

4. **In v2.0, 532 languages have more than one inventory, because several linguists have described the same language and, of course, they do not always agree. Before computing anything else, you need one inventory per language. For this lab, adopt the following convention: for each Glottocode, keep only the inventory with the highest InventoryID (i.e. treat the most recently added description as the reference one), and discard the others.**

Do you think this is a reasonable choice? What alternatives could you have used instead (a randomly chosen inventory, a fixed preference for a given source, merging the inventories), and what would each of them cost you? Keep in mind that every result in this lab depends on this decision.

This choice is kind of reasonable. Keeping the most recently added inventory is simple and easy to reproduce. However, the most recently added description is not always the most accurate or complete one.

Alternatives that we could use instead:
A random inventory may give different results depending on the selection.
Preferring one source can make the data more consistent, but it may introduce source bias and may not cover all languages.
Merging inventories keeps more phonemes, but it creates an artificial inventory that was not actually described by a linguist, which might not be very correct.

These choices can affect the number of vowels, phoneme distributions, and the conclusions about linguistic universals.

For each Glottocode, the inventory with the highest `InventoryID` and discarded the others ⬇️

Code block

```
latest_inventory = df.groupby("Glottocode")["InventoryID"].max().reset_index()
df_selected = df.merge(latest_inventory,on=["Glottocode", "InventoryID"])
```

5. **Measure how much the descriptions disagree: for a few languages described several times, compare the inventories. Do they have the same size? The same phonemes?**

Code block

```
multi_inventory_languages = (df.groupby("Glottocode")["InventoryID"].nunique())
multi_inventory_languages = multi_inventory_languages[
multi_inventory_languages > 3]
sample_languages = multi_inventory_languages.sample(5, random_state=42).index

for glottocode in sample_languages:
    language_data = df[df["Glottocode"] == glottocode]
    inventories = [set(group["Phoneme"])
        for _, group in language_data.groupby("InventoryID")]
    sizes = [len(x) for x in inventories]
    print(
        language_data["LanguageName"].iloc[0],
        "| sizes:", sizes,
        "| same phonemes:", all(x == inventories[0] for x in inventories)
    )
```

> French | sizes: [40, 37, 36, 34] | same phonemes: False
>
> Udmurt | sizes: [36, 36, 37, 33] | same phonemes: False
>
> fulfulde | sizes: [34, 36, 47, 39] | same phonemes: False
>
> Igbo | sizes: [65, 59, 36, 35] | same phonemes: False
>
> Bidyara | sizes: [20, 17, 18, 21, 19] | same phonemes: False

The inventories do not have the same size or the same phonemes. For example, French has inventory sizes of 40, 37, 36 and 34 phonemes. Udmurt has 36, 36, 37 and 33 phonemes. Fulfulde, Igbo and Bidyara also show differences in inventory size. In all five languages, the phoneme sets are different between inventories.

This shows that different descriptions of the same language can disagree both in the number of phonemes and in which phonemes are included.

6. **Compare the distribution of inventory sizes across the values of Source. Do the sources agree? What would it mean for your conclusions if one source systematically reported larger inventories than another?**

Code block

```
inventory_sizes = (
    df.groupby(["Source", "InventoryID"]).size()
      .reset_index(name="InventorySize")
)
source_stats = (inventory_sizes.groupby("Source")["InventorySize"].describe())
print(source_stats)
```

Source	count	mean	std	min	25%	50%	75%	max
aa	203.0	39.724138	8.771734	22.0	34.0	38.0	44.00	82.0
ea	390.0	43.292308	14.319767	19.0	34.0	40.0	49.75	133.0
er	392.0	24.038265	4.836568	16.0	21.0	23.0	26.00	44.0
gm	460.0	41.917391	14.541718	18.0	33.0	39.0	47.00	161.0
ph	389.0	34.089974	11.452035	14.0	25.0	33.0	41.00	90.0
ra	100.0	42.610000	8.491106	21.0	36.0	42.0	48.00	62.0
saphon	355.0	25.495775	6.184714	11.0	21.0	25.0	29.00	51.0
spa	197.0	38.406091	12.957055	17.0	28.0	37.0	45.00	94.0
upsid	451.0	30.966741	11.554364	11.0	23.5	29.0	36.00	141.0
uz	83.0	44.686747	14.378955	21.0	34.5	41.0	55.50	74.0

The distribution of inventory sizes is different across sources. Some sources tend to report larger inventories, while others report smaller ones. This suggests that source bias may affect the results, because the differences may come partly from the way different sources describe languages, not only from the languages themselves.

7. **State explicitly what you will count as a vowel. For this lab, adopt the following criterion: a vowel is any row with SegmentClass == "vowel", and you keep it regardless of its Marginal value (TRUE, FALSE or NA). Write this criterion down — you will need it in every later section — and discuss the advantages and disadvantages of this choice compared with the alternative of excluding all Marginal == TRUE phonemes, and with the alternative of excluding rows where Marginal is NA.**

   With these conventions fixed, we can now compute some basic descriptive statistics.

For this lab, I will count every row with SegmentClass == "vowel" as a vowel, regardless of whether Marginal is TRUE, FALSE, or NA. This can keep all possible vowels and avoids missing possible vowels because of differences in Marginal annotation, and it keeps more information.

If we exclude Marginal == TRUE, we can focus on the core vowel system and avoid counting rare or marginal phonemes. But we may lose real vowels that are less common.

If we exclude Marginal == NA, the data will be cleaner, and we do not mix phonemes with unknown status with phonemes whose status is known. This can be useful if we want to compare marginal and non-marginal phonemes. However, we may lose real vowels and make some vowel inventories artificially smaller.

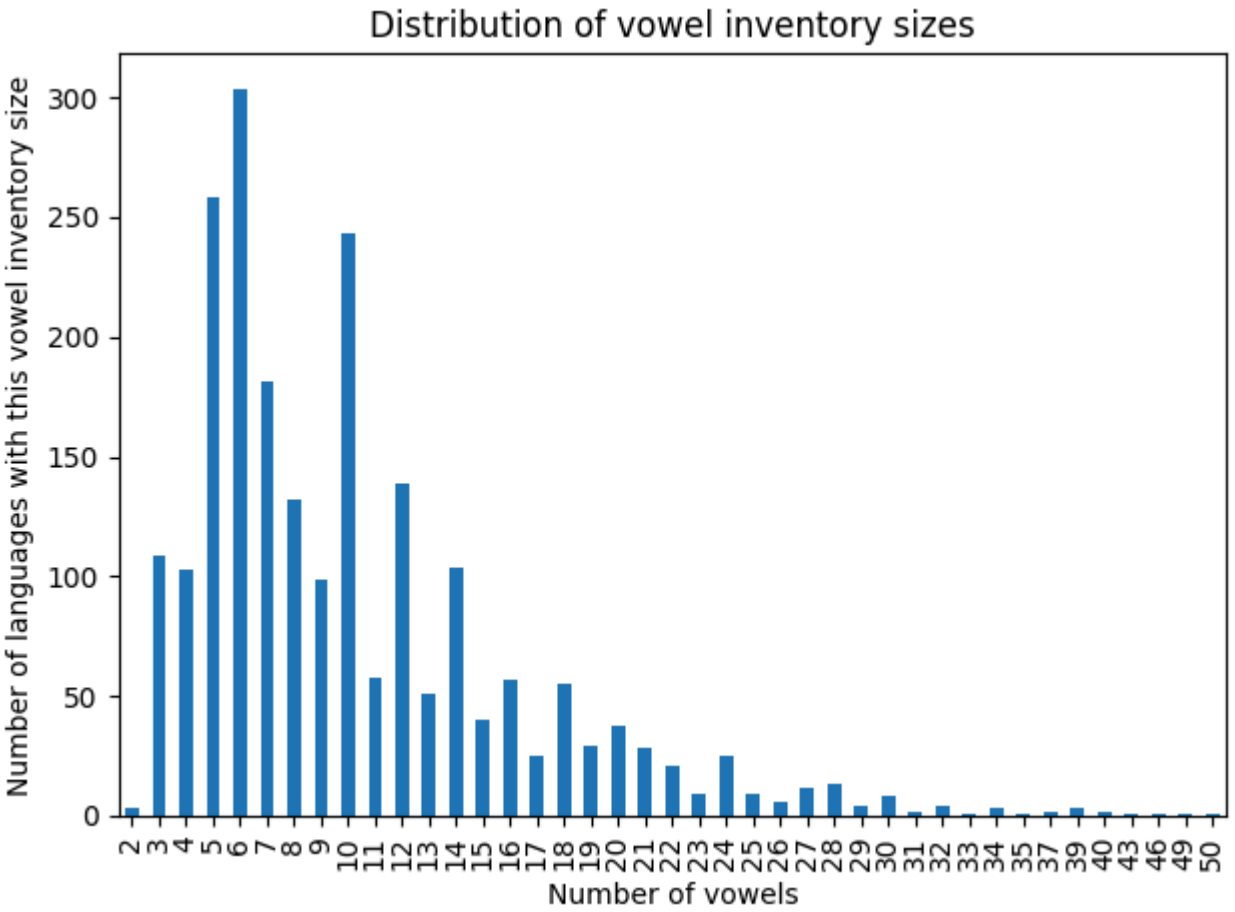
8. **Using the inventory-selection and vowel-definition conventions you have just settled on, compute the average number of vowels per language and the distribution of vowel inventory sizes.**

Code block

```
vowels = df_selected[df_selected["SegmentClass"] == "vowel"] # keep only vowels
vowel_counts = vowels.groupby("Glottocode").size() # vowel inventory size for each lang
print("Average number of vowels:", vowel_counts.mean())
distribution = vowel_counts.value_counts().sort_index()

distribution.plot(kind="bar")
plt.xlabel("Number of vowels")
plt.ylabel("Number of languages with this vowel inventory size")
plt.title("Distribution of vowel inventory sizes")
plt.tight_layout()
plt.show()
```

The average number of vowels per language is 10.192765567765568.
The distribution of vowel inventory sizes is shown in the figure below.



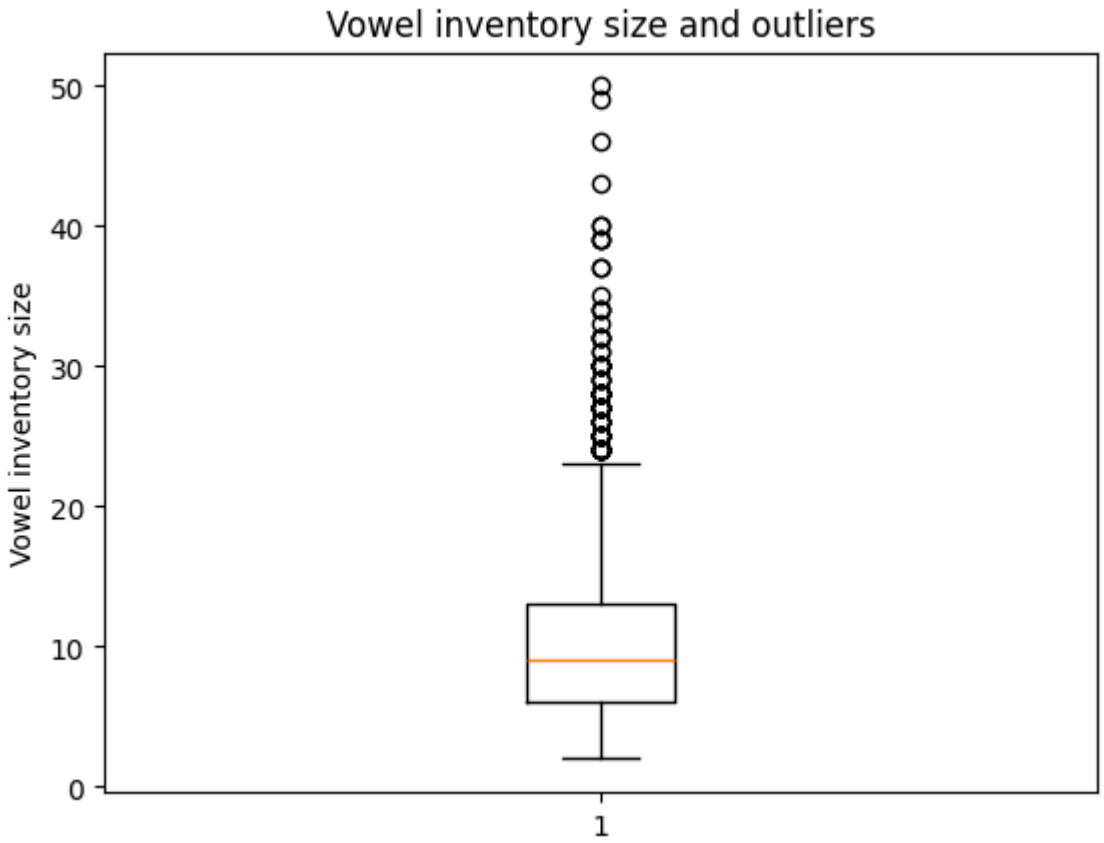
9. **Explain why it is interesting to examine these statistics.**

The average shows the general size of vowel inventories across languages. The distribution gives more details, such as the minimum and maximum sizes, where most languages are concentrated, how much languages differ, and whether there are unusual values. It also shows whether the average is representative of most languages.

10. **Identify outliers using the interquartile range (IQR) method. Briefly explain the method and comment on your findings.**

Code block

```
Q1, Q3 = vowel_counts.quantile([0.25, 0.75])
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = vowel_counts[(vowel_counts < lower) | (vowel_counts > upper)]
```



The IQR method uses the first quartile (Q1) and the third quartile (Q3). Here, Q1 = 6 and Q3 = 13, so the IQR is 7. Values below Q1 - 1.5 × IQR or above Q3 + 1.5 × IQR are considered outliers.

The lower bound is -4.5 and the upper bound is 23.5. Since a vowel inventory size cannot be negative, there are no lower outliers. I found 99 languages with more than 23.5 vowels, so these languages have unusually large vowel inventories.

3 Testing Linguistic Universals

The basic idea of this part is to use the data to test whether the proposed universals are actually

observed across the sample of languages in the PHOIBLE database.

11. **Identify the three most frequent vowels (those occurring in the largest number of languages).**

Code block

```
top_vowels = (vowels.groupby("Phoneme")["Glottocode"]
                    .nunique().sort_values(ascending=False).head(3))
```

The three most frequent vowels:　　i 2059 ;　　u 1966 ;　a 1922

12. **Compute the proportion of languages that contain each vowel of the basic vowel triangle [i], [a], [u], and discuss whether this supports the first universal.**

Code block

```
total_languages = df_selected["Glottocode"].nunique()
for vowel in ["i", "a", "u"]:
    count = vowels[vowels["Phoneme"] == vowel]["Glottocode"].nunique()
    proportion = count / total_languages
    print(vowel, proportion)

phoneme_sets = vowels.groupby("Glottocode")["Phoneme"].apply(set)
all_three = phoneme_sets.apply(lambda x: {"i", "a", "u"}.issubset(x))
print("i+a+u:", all_three.sum() / total_languages)
```

> i 0.9427655677655677
>
> a 0.88003663003663
>
> u 0.9001831501831502
>
> i+a+u: 0.7976190476190477

The individual proportions are high: [i] occurs in about 94.3% of languages, [a] in 88.0%, and [u] in 90.0%. Only about 79.8% of languages contain all three vowels [i], [a], and [u] at the same time.

This shows that the three vowels are very common, but the strict version of the first universal is not supported by all languages. This is only a first test, since the universal also includes “close equivalents”.

13. **The first universal is stated for [i], [a], [u] “or close equivalents”. Give an explicit criterion for deciding what counts as a close equivalent (does a nasalised, long or otherwise modified [i] count as an [i]?), and recompute the proportions under your criterion. Does the conclusion change? This is a good illustration of how much a typological result can depend on a decision that looks purely technical.**

Code block

```
#13
import unicodedata
#Convert the ‘vowel with a diacritical mark’ back to its basic vowel letter
def clean_vowel(x):
    x = unicodedata.normalize("NFD", x)
    x = "".join(c for c in x if unicodedata.category(c) != "Mn")
    return x.replace("ː", "").replace("ˑ", "")
```

```
8
9   vowels = vowels.copy()
10  vowels["base"] = vowels["Phoneme"].apply(clean_vowel)
11
12  for v in ["i", "a", "u"]:
13      count = vowels[vowels["base"] == v]["Glottocode"].nunique()
14      print(v, count / total_languages)
15
16  base_sets = vowels.groupby("Glottocode")["base"].apply(set)
17  all_three_close = base_sets.apply(lambda x: {"i", "a", "u"}.issubset(x))
18  print("i+a+u (close equivalents):",
19        all_three_close.sum() / total_languages)
```

> i 0.9716117216117216
>
> a 0.9317765567765568
>
> u 0.9290293040293041
>
> i+a+u (close equivalents): 0.8644688644688645

I consider a vowel a close equivalent of [i], [a], or [u] if it has the same basic vowel symbol but only differs by diacritics such as nasalisation or vowel length. For example, [ĩ] and [iː] count as [i]. However, vowels with a different basic symbol, such as [ɪ] or [y], are not included.

With this criterion, the proportions are higher: about 97.2% for [i], 93.2% for [a], and 92.9% for [u]. About 86.4% of languages contain all three vowels or their close equivalents. The proportions under this criterion are higher. The conclusion does not change. This generally supports the idea that "**virtually** all languages possess the basic vowel triangle [i], [a], [u] (or close equivalents)". This is not an absolute universal, because there are still some exceptions.

14. An absolute universal admits no exception; a statistical universal only a few. Count the languages that violate the first universal and inspect a handful of them. Do the data support an absolute universal, a statistical tendency, or neither?

Code block

```
1   language_vowels = (vowels.groupby("Glottocode")["base"].apply(set))
2   violations = language_vowels[~language_vowels.apply(lambda x: {"i", "a",
    "u"}.issubset(x))]
3   print("Number of violations:", len(violations), 1 -
    (len(violations)/len(language_vowels)))
4
5   examples = violations.tail(3)
6   for code, phonemes in examples.items():
```

```
        name = df_selected.loc[df_selected["Glottocode"] ==
    code,"LanguageName"].iloc[0]
        missing = {"i", "a", "u"} - phonemes
        print(name)
        print("Missing:", missing)
        print("Vowels:", phonemes)
        print()
```

> Number of violations: 296 0.8644688644688645
>
> Euchee; Yuchi
>
> Missing: {'a'}
>
> Vowels: {'æ', 'i', 'o', 'u', 'e', 'ɑ'}
>
> zande
>
> Missing: {'i', 'u'}
>
> Vowels: {'ɔ', 'ɪ', 'ʊ', 'a', 'ɛ'}
>
> zulgo
>
> Missing: {'i', 'u'}
>
> Vowels: {'a', 'ə'}

The data does not support an absolute universal, because there are 296 exceptions, and about 86.5% of the languages still follow the pattern, so the result supports a strong statistical tendency.

Inspection：For example, Yuchi lacks [a] but has [æ] and [ɑ], while Zande lacks [i] and [u] but has [ɪ] and [ʊ]. These examples show that some violations are related to how “close equivalents” are defined.

15. **Are the exceptions genuine counter-examples, or artefacts of the description (transcription choices, phonemes marked as marginal, a single idiosyncratic source)? Check a few of them in the database itself.**

Code block

```
languages_selected = ["Euchee; Yuchi", "zande", "zulgo"]
check = df[(df["LanguageName"].isin(languages_selected)) &
           (df["SegmentClass"] == "vowel")][["LanguageName", "InventoryID",
"Phoneme", "Marginal", "Source"]]
```

```
print(check.to_string(index=False))
```

```
LanguageName  InventoryID Phoneme Marginal Source
      zande          849       a    False     aa
      zande          849       ɛ    False     aa
      zande          849       ɪ    False     aa
      zande          849       ɔ    False     aa
      zande          849       ʊ    False     aa
      zulgo          851       a    False     aa
      zulgo          851       ə    False     aa
Euchee; Yuchi        1074       æ    False     ph
Euchee; Yuchi        1074       æ̃    False     ph
Euchee; Yuchi        1074       ɑ    False     ph
Euchee; Yuchi        1074       ɑ̃    False     ph
Euchee; Yuchi        1074       e    False     ph
Euchee; Yuchi        1074       ẽ    False     ph
Euchee; Yuchi        1074       i    False     ph
Euchee; Yuchi        1074       ĩ    False     ph
Euchee; Yuchi        1074       o    False     ph
Euchee; Yuchi        1074       õ    False     ph
Euchee; Yuchi        1074       u    False     ph
```

The three examples are not caused by marginal phonemes, since all their vowels are marked as non-marginal.

Some exceptions may be related to transcription choices or to the definition of “close equivalents”. Yuchi lacks [a], but has [æ] and [ɑ], while Zande lacks [i] and [u], but has [ɪ] and [ʊ]. These are therefore not very strong counter-examples.Zulgo is different because its inventory contains only [a] and [ə], so it appears to be a more genuine counter-example. Each of these 3 languages is represented by only one inventory from one source, so source-specific descriptions cannot be completely excluded.

3.1 The Second Universal: An Implication

The second universal has a different logical form: it does not state that some vowel is frequent, but that the presence of a marked vowel implies the presence of its simpler counterpart.

16. **Build the list of languages having at least one nasal vowel, at least one long vowel, and at least one front rounded vowel ([y], [ø], . . .).**

Code block

```
 # nasal vowels
nasal_languages = vowels[vowels["nasal"] == "+"]["Glottocode"].unique()

# long vowels
long_languages = vowels[vowels["long"] == "+"]["Glottocode"].unique()

# front rounded vowels
front_rounded_languages = vowels[(vowels["front"] == "+") &(vowels["round"] == "+")
                                 ]["Glottocode"].unique()

print("Nasal:", len(nasal_languages))
print("Long:", len(long_languages))
print("Front rounded:", len(front_rounded_languages))

```

> Nasal: 473
>
> Long: 821
>
> Front rounded: 173

17. **Test the implication for each of these three cases: does a language with a nasal vowel also have the corresponding oral vowel? Does a language with a long vowel also have the corresponding short one? Does a language with [y] also have [i] and [u]? Report, in each case, the proportion of languages satisfying the implication and the number of exceptions.**

Code block

```
# all phonological feature columns
feature_cols = vowels.loc[:, "tone":"click"].columns.tolist()

# nasal -> oral
nasal_results = []
for code in nasal_languages:
    data = vowels[vowels["Glottocode"] == code]
    nasal_vowels = data[data["nasal"] == "+"]
```

```
    oral_vowels = data[data["nasal"] == "-"]
    other_features = [f for f in feature_cols if f != "nasal"]

    result = all(
        any((oral_vowels[other_features] == row[other_features]).all(axis=1))
        for _, row in nasal_vowels.iterrows()
    )
    nasal_results.append(result)

print("Nasal:",sum(nasal_results) / len(nasal_results),
      "exceptions:", nasal_results.count(False))

# long -> non-long
long_results = []
for code in long_languages:
    data = vowels[vowels["Glottocode"] == code]
    long_vowels = data[data["long"] == "+"]
    short_vowels = data[data["long"] == "-"]
    other_features = [f for f in feature_cols if f != "long"]

    result = all(
        any((short_vowels[other_features] == row[other_features]).all(axis=1))
        for _, row in long_vowels.iterrows()
    )
    long_results.append(result)

print("Long:",sum(long_results) / len(long_results),
      "exceptions:", long_results.count(False))

# y -> i and u
y_languages = vowels[vowels["Phoneme"] == "y"]["Glottocode"].unique()
y_results = []
for code in y_languages:
    inv = set(vowels[vowels["Glottocode"] == code]["Phoneme"])
    y_results.append("i" in inv and "u" in inv)

print("y:",sum(y_results) / len(y_results),
      "exceptions:", y_results.count(False))
```

> Nasal: 0.8837209302325582 exceptions: 55
>
> Long: 0.7758830694275274 exceptions: 184
>
> y: 0.8981481481481481 exceptions: 11

For nasal and long vowels, I define the corresponding simpler vowel as a vowel with the same phonological features, except that nasal or long changes from + to -. For [y], I check whether the

same language also has both [i] and [u].

For nasal vowels, 88.4% of the languages also have the corresponding oral vowel, with 55 exceptions.

For long vowels, 77.6% also have the corresponding non-long vowel, with 184 exceptions.

For [y], 89.8% of the languages also have both [i] and [u], with 11 exceptions.

The implication is strongest for [y], followed by nasal vowels, and weakest for long vowels.

18. **An implicational universal "if A then B" corresponds to a specific pattern in a 2 × 2 table counting languages. Which cell of that table must be (nearly) empty? Why is this pattern not the same thing as a symmetric association between A and B?**

The cell where A is present but B is absent should be **(nearly)** empty. These are the exceptions to A → B.

The relation is not symmetric. The implication has a direction. A → B does not mean B → A. One language can have B without having A.

19. **Compare the two universals: which one is better supported by PHOIBLE? Which one is more informative about the structure of vowel systems?**

The first universal shows a strong overall tendency, with about 86.4% of languages having [i], [a], [u] or close equivalents. The second universal is more variable: the support depends on the type of implication.

The second universal is more informative about the structure of vowel systems because it shows a relationship between marked vowels and their simpler counterparts, rather than only showing which vowels are common.

3.2 Association Between Pairs of Phonemes

We will now investigate whether two phonemes tend to occur together in the same languages more often than would be expected by chance. For each pair (A, B) of phonemes:

20. **Construct a 2 × 2 contingency table indicating, for each language in the dataset:**
    - **whether it has phoneme A,**
    - **whether it has phoneme B.**

The four cells of the table will then correspond to: (A present & B present), (A present & B absent), (A absent & B present), (A absent & B absent).

Code block

```
A = "y"
B = "i"
language_phonemes = df_selected.groupby("Glottocode")["Phoneme"].apply(set)
has_y = language_phonemes.apply(lambda x: "y" in x)
has_i = language_phonemes.apply(lambda x: "i" in x)
table = pd.crosstab(has_y, has_i)
table.index.name = "y"
table.columns.name = "i"
print(table)
```

I chose the phoneme pair [y] and [i] as an example. The 2×2 contingency table is:

y \ i	FALSE	TRUE
FALSE	119	1957
TRUE	6	102

21. **Compute a χ^2 test of independence on this table. Does the result suggest that the two phonemes occur independently, or that there is a statistically significant association?**

Code block

```
from scipy.stats import chi2_contingency
chi2, p, dof, expected = chi2_contingency(table)
print("chi2 =", chi2, "; p =", p)
```

chi2 = 0.0 ; p = 1.0

The chi-square test gives $\chi^2 = 0.0$ and $p = 1.0$. Since $p > 0.05$, there is NO statistically significant association between [y] and [i]. The result suggests that there is no evidence of a significant association.

22. **Compute the mutual information between the two phonemes, defined as**

$$I(A;B) = \sum_{a \in \{0,1\}} \sum_{b \in \{0,1\}} p(a,b) \log \frac{p(a,b)}{p(a)p(b)}$$

where p(a,b) is the proportion of languages with the configuration (a,b), and p(a) and p(b) are the marginal probabilities of each phoneme. Interpret the value. Mutual information is always non-negative: a value close to zero indicates independence, but a high value does not tell you whether the two phonemes tend to co-occur or to exclude each other. How can

you determine the direction of the association (e.g. by comparing p(1,1) with p(a=1) p(b=1))?

Code block

```
from sklearn.metrics import mutual_info_score
mi = mutual_info_score(has_y, has_i)
p1_1 = (has_y & has_i).mean()
expected = has_y.mean() * has_i.mean()
print("MI =", mi)
print("p(1,1) =", p1_1)
print("p(y=1)p(i=1) =", expected)
```

MI = 1.370768209975337e-06

p(1,1) = 0.046703296703296704

p(y=1)p(i=1) = 0.046620275329066534

The mutual information is 0.00000137, which is very close to zero. This suggests that [y] and [i] are almost independent. The observed (p(1,1)) is slightly higher than (p(y=1)p(i=1)), so there is a very small tendency for them to co-occur.

23. **Compare the results of the two methods (χ^2 and mutual information). Do they lead to similar conclusions about the relationship between the two phonemes?**

The two methods give similar results. The χ^2 test finds no significant association, and the mutual information is very close to zero. Both suggest that [y] and [i] are almost independent.

24. **The χ^2 test is only valid if the expected count in each cell is large enough (a common rule of thumb is 5). Check this condition for a pair involving a rare phoneme. When it fails, use Fisher's exact test instead and compare the two p-values.**

Code block

```
freq = (vowels.groupby("Phoneme")["Glottocode"].nunique().sort_values())
print(freq[(freq > 1) & (freq <= 10)].head(5))
has_ay = language_phonemes.apply(lambda x: "ay" in x)
has_i = language_phonemes.apply(lambda x: "i" in x)
rare_table = pd.crosstab(has_ay, has_i)
rare_table.index.name = "ay"
rare_table.columns.name = "i"

print(rare_table)
chi2, p_chi, dof, expected = chi2_contingency(rare_table)
```

```
print(expected)
print("Chi-square p =", p_chi)

from scipy.stats import fisher_exact
oddsratio, p_fisher = fisher_exact(rare_table)
print("Fisher p =", p_fisher)
```

```
Phoneme
ʌ̃ː  2
aːu  2
aĩ  2
ʊ̯  2
ay  2
Name: Glottocode, dtype: int64
i    False  True
ay
False   125  2057
True     0    2


[[1.24885531e+02 2.05711447e+03]
 [1.14468864e-01 1.88553114e+00]]
Chi-square p = 1.0
Fisher p = 1.0
```

I chose the rare phoneme [ay] with [i]. Two expected counts are below 5 (0.11 and 1.89), so the condition for the χ^2 test is not satisfied. Therefore, I used Fisher's exact test instead. The test gives p = 1.0, so there is NO significant association between [ay] and [i].

25. **Suppose you now test all pairs of phonemes. How many tests does that represent? At the 5 % level, how many significant results would you expect by chance alone if no pair were associated? Explain, in your own words, why testing many pairs like this is a problem, and why a correction is needed. Then apply a correction for multiple testing (Bonferroni, or Benjamini–Hochberg) and report how many associations survive it.**

Code block

```
# 25
```

```
2
3  import numpy as np
4  from scipy.stats import chi2, false_discovery_control
5  # language × phoneme
6  presence = pd.crosstab(df_selected["Glottocode"],df_selected["Phoneme"]) > 0
7
8  X = presence.to_numpy(dtype=np.int32)
9  n_languages = X.shape[0]
10 n_phonemes = X.shape[1]
11
12 # number of all phoneme pairs
13 n_tests = n_phonemes * (n_phonemes - 1) // 2
14 expected_false_positives = n_tests * 0.05
15
16 print("Number of phonemes:", n_phonemes)
17 print("Number of tests:", n_tests)
18 print("Expected false positives:", expected_false_positives)
19
20 # number of languages containing each phoneme
21 freq = X.sum(axis=0)
22 # number of languages containing both phonemes
23 both = X.T @ X
24 # only keep each pair once: upper triangle
25 i, j = np.triu_indices(n_phonemes, k=1)
26
27 n11 = both[i, j]          # A present, B present
28 n10 = freq[i] - n11       # A present, B absent
29 n01 = freq[j] - n11       # A absent, B present
30 n00 = n_languages - n11 - n10 - n01
31
32 # Pearson chi-square for every 2x2 table
33 numerator = n_languages * (n11 * n00 - n10 * n01) ** 2
34 denominator = (
35     (n11 + n10) *
36     (n01 + n00) *
37     (n11 + n01) *
38     (n10 + n00)
39 )
40
41 chi2_values = numerator / denominator
42 p_values = chi2.sf(chi2_values, df=1)
43
44 # significant before correction
45 print("Significant before correction:",np.sum(p_values < 0.05))
46
47 # Benjamini-Hochberg correction
48 p_corrected = false_discovery_control(p_values,method="bh")
```

```
print("Significant after BH:",np.sum(p_corrected < 0.05))
```

> Number of phonemes: 2609
>
> Number of tests: 3402136
>
> Expected false positives: 170106.80000000002
>
> Significant before correction: 121374
>
> Significant after BH: 89708

false_discovery_control — SciPy v1.18.0 Manual
statsmodels.stats.multitest.fdrcorrection — statsmodels 0.15.1 (+75)

Number of tests: 3402136

At the 5% significance level, if none of the phoneme pairs were truly associated, we would expect about 170,107 false positives by chance alone.

Why testing many pairs like this is a problem, and why a correction is needed.

If we run only one test with a 5% significance level, there is still a 5% chance of getting a "significant" result just by chance, even when the two phonemes are actually not associated, and this is what we call "**false positive**". If we run **3,402,136 tests and** none of the phoneme pairs are truly associated, about **170,107 results** would still be expected to appear significant just by chance.

So, when we run many tests, we can get many false significant results. This does not mean that the phonemes are really associated. It happens because there are so many tests.

We need to control the overall chance of making errors, not only the 5% error rate of each individual test. A multiple-testing correction makes the significance threshold stricter, so the associations that remain significant are more reliable.

Then apply a correction for multiple testing (Bonferroni, or Benjamini–Hochberg) and report how many associations survive it.

For all phoneme pairs, I first calculated a p-value based on their presence or absence across languages. Before correction, 121,374 associations had $p < 0.05$. I then applied the Benjamini–Hochberg correction to control the false discovery rate. After correction, 89,708 associations remained significant at the 5% level. Therefore, 89,708 associations survive the Benjamini–Hochberg correction.

26. **With more than 2,000 languages, even a negligible association becomes statistically significant. Compute an effect size (ϕ coefficient or Cramér's V) for your pairs and rank them by effect size rather than by p-value. Is it the same ranking? Which of the two rankings is linguistically interesting?**

Code block

```
names = np.array(presence.columns)
phi = (n11 * n00 - n10 * n01) / np.sqrt(
    (n11 + n10) * (n01 + n00) * (n11 + n01) * (n10 + n00))

result = pd.DataFrame({
    "phoneme1": names[i],
    "phoneme2": names[j],
    "p": p_values,
    "phi": abs(phi)
})

print("Top 10 by p-value:")
print(result.sort_values("p").head(10))
print()
print("Top 10 by effect size:")
print(result.sort_values("phi", ascending=False).head(10))
```

Top 10 by p-value:

	phoneme1	phoneme2	p	phi
1069112	i□ɸ	ũo	0.0	1.0
2989472	ɔy̯ː	ɲː	0.0	1.0
643251	d□□	z□	0.0	1.0
1303906	kʲː	βʲː	0.0	1.0
1877222	õə̯□	ts□	0.0	1.0
1473382	mbˠ	ŋh	0.0	1.0
1223672	k‡h	oᴱ	0.0	1.0
2659583	xʲː	ʰkʲ	0.0	1.0
2659601	xʲː	ʰtʲ	0.0	1.0
2659598	xʲː	ʰtɕ	0.0	1.0

Top 10 by effect size:

	phoneme1	phoneme2	p	phi
736754	ĕ	ɛa̯	0.0	1.0
3402051	ⁿts□	ⁿ□□□	0.0	1.0
3402046	ⁿts□	ⁿ□□	0.0	1.0

> 3402038 ⁿt͡s□□ ⁿ□□□ 0.0 1.0
>
> 3402033 ⁿt͡s□□ ⁿ□□ 0.0 1.0
>
> 736760 ĕ ɛɐ 0.0 1.0
>
> 1223308 k‡h k‡kx’ 0.0 1.0
>
> 3402074 ⁿɖɽ ⁿʈʈʰ 0.0 1.0
>
> 563346 d□ □□ 0.0 1.0
>
> 1697485 ng□ ps□ 0.0 1.0

https://stats.oarc.ucla.edu/sas/output/proc-freq/

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.contingency.association.html?

https://pmc.ncbi.nlm.nih.gov/articles/PMC3444174/

I used the ϕ coefficient as the effect size because each phoneme pair forms a 2×2 presence/absence table. I ranked the pairs by the absolute value of ϕ.

The ranking by effect size is not the same as the ranking by p-value. Several pairs have p-values displayed as 0 and $\phi = 1$, so the exact order among these pairs is not very informative.

The effect-size ranking is more linguistically interesting because the p-value mainly tells us whether an association is statistically significant, while ϕ tells us how strong the association is. With more than 2,000 languages, even a weak association can have a very small p-value.

27. **Build a null model to calibrate your results: shuffle the inventories at random while keeping both the inventory sizes and the overall frequency of each phoneme unchanged, then recompute your statistics on the shuffled data. How many of your associations remain when compared against this baseline rather than against the independence assumption?**

Code block

```
# 27 - shuffle inventories while keeping:
#  each language's inventory size unchanged
#  each phoneme's overall frequency unchanged
def shuffle(X: np.ndarray, seed: int = 42) -> np.ndarray:
    """
    X:  Binary language × phoneme matrix.
        Rows = languages, columns = phonemes.
        1 = phoneme present, 0 = absent.
    return: Y. Shuffled binary matrix with the same shape as X.
    """
    rng = np.random.default_rng(seed)
    # phonemes present in each language
```

```
    rows = [set(np.flatnonzero(row)) for row in X]

    # randomly exchange phonemes between two languages
    for _ in range(20000):
        r1, r2 = rng.choice(len(rows), 2, replace=False)
        only1 = list(rows[r1] - rows[r2])
        only2 = list(rows[r2] - rows[r1])
        if only1 and only2:
            x = rng.choice(only1)
            y = rng.choice(only2)
            rows[r1].remove(x)
            rows[r1].add(y)
            rows[r2].remove(y)
            rows[r2].add(x)
    # convert back to a binary matrix
    Y = np.zeros_like(X)
    for r, sounds in enumerate(rows):
        Y[r, list(sounds)] = 1
    return Y

keep = p_corrected < 0.05
a, b = i[keep], j[keep]
real_effect = abs(phi[keep])

# shuffled null model
Y = shuffle(X, seed=42)

freq = Y.sum(axis=0)
both = Y.T @ Y

n11_null = both[a, b]
n10_null = freq[a] - n11_null
n01_null = freq[b] - n11_null
n00_null = len(Y) - n11_null - n10_null - n01_null

null_effect = abs(
    (n11_null * n00_null - n10_null * n01_null) /
    np.sqrt((n11_null + n10_null) * (n01_null + n00_null) * (n11_null +
n01_null) * (n10_null + n00_null)))

# real association is larger than the shuffled baseline
remain = real_effect > null_effect
print("Associations before null model:", len(a))
print("Associations remaining after comparison with null model:", remain.sum())
```

> Associations before null model: 89708

> Associations remaining after comparison with null model: 52752

I created a null model by randomly shuffling the phoneme inventories, while keeping the inventory size of each language and the overall frequency of each phoneme unchanged. I used a fixed random seed (seed = 42) to make the result reproducible.

Then I compared the absolute ϕ values from the real data with the ϕ values from the shuffled data.

Before using the null model, 89,708 associations remained after the BH correction. After comparing them with the shuffled baseline, 52,752 associations were stronger than in the random data.

So, 52,752 associations have a larger absolute ϕ than in this shuffled null-model baseline.

28. **What can you conclude?**

- Statistical significance ≠ strong association. With more than 2,000 languages, even weak associations can have very small p-values, so effect size is also important.
- Some associations may come from the structure of the data. After BH correction, 89,708 associations remained, but only 52,752 were stronger than the shuffled null baseline.
- The inventories are still not completely random. Many associations remained after comparison with the null model, suggesting some real structure in phoneme co-occurrence or avoidance.
- Some limitations: association does not imply causation; rare phonemes can produce large effect sizes; the null model only controls inventory size and phoneme frequency, not language family or geographical area.
- Only one shuffle was used, so the result may change with a different shuffle.

4 Are Languages Independent Observations?

Every test you have run so far treats each language as an independent observation. This is, however, false: languages are related to one another, either genealogically (they descend from a common ancestor) or areally (they are spoken by neighbouring communities and influence one another). A property shared by fifty Indo-European languages may be a single historical accident inherited fifty times, rather than fifty independent pieces of evidence for a universal. This difficulty is known in typology as Galton's problem, and it affects every result obtained in the previous sections.

29. **Using the genealogical information available for the languages of the sample (language family, and macroarea), determine which families and which areas are over-represented in PHOIBLE. Produce a table or a figure.**

The phoible.csv file does not contain language family or macroarea information. PHOIBLE Online (https://phoible.org/languages) provides these language-level metadata, and the PHOIBLE FAQ explains that they can be linked to the inventory data using Glottocodes. https://phoible.org/faq?

Code block

```
# language metadata
url = "https://raw.githubusercontent.com/cldf-datasets/phoible/master/cldf/languages.csv"
meta = pd.read_csv(url)

# one row per language in our sample
languages = df_selected[["Glottocode", "LanguageName"]].drop_duplicates("Glottocode")

# add family and macroarea
languages = languages.merge(
    meta[["Glottocode", "Family_Name", "Macroarea"]],
    on="Glottocode",how="left"
)

# family distribution
family_table = (
    languages["Family_Name"].value_counts(dropna=False).reset_index())
family_table.columns = ["Family", "Languages"]

# macroarea distribution
area_table = (
    languages["Macroarea"].value_counts(dropna=False).reset_index())
area_table.columns = ["Macroarea", "Languages"]

print("Most represented families:")
print(family_table.head(10))

print("\nMacroareas:")
print(area_table)
```

Most represented families:

	Family	Languages
0	Atlantic-Congo	420
1	Pama-Nyungan	253
2	Indo-European	146

```
3   Austronesian     110
4   Afro-Asiatic     107
5   Sino-Tibetan      99
6            NaN      93
7         Tupian      50
8       Arawakan      43
9         Mande       43


Macroareas:
     Macroarea  Languages
0       Africa      714
1      Eurasia      448
2    Australia      343
3  South America      332
4     Papunesia      183
5  North America      141
6            NaN      23
```

The most represented families are Atlantic-Congo (420 languages), Pama-Nyungan (253) and Indo-European (146).

For macroareas, Africa is the most represented (714 languages), followed by Eurasia (448), Australia (343) and South America (332).

So, some families and areas are much more represented than others in the PHOIBLE sample.

30. **Draw a genealogically balanced sample — for example one language per family, chosen at random — and recompute the proportions of the previous section on this sample. Do your conclusions hold?**

Code block

```
# one language per family
info = languages.copy()
info["family"] = info["Family_Name"].fillna(info["Glottocode"])
balanced = (info.groupby("family", group_keys=False)
           .sample(1, random_state=42))
```

```
6
7  codes = balanced["Glottocode"]
8  sample_vowels = vowels[vowels["Glottocode"].isin(codes)].copy()
9  print("Languages in balanced sample:", len(codes))
```

> Languages in balanced sample: 269

Code block

```
1  # for the 1st universal
2  for sound in ["i", "a", "u"]:
3      count = sample_vowels[sample_vowels["base"] == sound]
   ["Glottocode"].nunique()
4      print(sound, count / len(codes))
5  sets = sample_vowels.groupby("Glottocode")["base"].apply(set)
6  print("i+a+u:",sets.apply(lambda x: {"i", "a", "u"}.issubset(x)).mean())
```

> i 0.966542750929368
>
> a 0.9405204460966543
>
> u 0.9033457249070632
>
> i+a+u: 0.8438661710037175

Code block

```
1  # for the 2nd universal
2  features = sample_vowels.loc[:, "tone":"click"].columns.tolist()
3  #Compute the proportion of languages satisfying a marked-vowel implication
4  def implication(feature: str) -> float:
5      # feature: phonological feature to test, e.g. "nasal" or "long"
6      # return: proportion of languages satisfying the implication
7      langs = sample_vowels[sample_vowels[feature] == "+"]["Glottocode"].unique()
8      result = []
9      other = [f for f in features if f != feature]
10
11     for code in langs:
12         data = sample_vowels[sample_vowels["Glottocode"] == code]
13         marked = data[data[feature] == "+"]
14         simple = data[data[feature] == "-"]
15         result.append(
16             all(any((simple[other] == row[other]).all(axis=1))for _, row in
   marked.iterrows()))
17     return sum(result) / len(result)
18
19 print("Nasal -> oral:", implication("nasal"))
```

```
print("Long -> non-long:", implication("long"))

phoneme_sets = (sample_vowels.groupby("Glottocode")["Phoneme"].apply(set))
with_y = phoneme_sets[phoneme_sets.apply(lambda x: "y" in x)]
print("y -> i+u:",with_y.apply(lambda x: "i" in x and "u" in x).mean())
```

> Nasal -> oral: 0.9818181818181818
>
> Long -> non-long: 0.7654320987654321
>
> y -> i+u: 0.8571428571428571

I created a genealogically balanced sample by randomly selecting one language per family with random_state = 42. This gave a sample of 269 languages.

For the first universal, the proportions were 96.7% for [i], 94.1% for [a], and 90.3% for [u]. About 84.4% of the languages contained all three vowels or their close equivalents.

For the second universal, 98.2% of the languages with nasal vowels also had the corresponding oral vowel, 76.5% of the languages with long vowels had the corresponding non-long vowel, and 85.7% of the languages with [y] also had both [i] and [u].

Overall, the exact percentages changed, but the general conclusions are similar. The first universal still shows a strong tendency, and the support for the second universal still depends on the type of implication.

31. **Repeat this sampling a large number of times and report the distribution of the proportions you obtain. How stable are your conclusions, and how would you report them honestly?**

Code block

```
# Q31 — repeat the balanced sampling 100 times
# Compute the proportion of languages satisfying a marked-vowel implication
# for one random genealogically balanced sample.
def sample_implication(sample_vowels: pd.DataFrame,feature: str) -> float:
    """
    sample_vowels: Vowel data from the current random sample.
    feature: Feature to test, e.g. "nasal" or "long".
    return: Proportion of languages satisfying the implication.
    """
    # phonological feature columns
    features = sample_vowels.loc[:, "tone":"click"].columns.tolist()
    # all features except the tested one
    other_features = [f for f in features if f != feature]
    # languages containing at least one marked vowel
    langs = sample_vowels[sample_vowels[feature] == "+"]["Glottocode"].unique()
```

```
    result = []

    for code in langs:
        # vowels of one language
        language_vowels = sample_vowels[sample_vowels["Glottocode"] == code]
        # marked vowels: e.g. nasal = "+"
        marked = language_vowels[language_vowels[feature] == "+"]
        # possible simpler counterparts: e.g. nasal = "-"
        simple = language_vowels[language_vowels[feature] == "-"]
        # check whether every marked vowel has a simpler counterpart
        ok = all(
            any((simple[other_features] == row[other_features]).all(axis=1))
            for _, row in marked.iterrows())
        result.append(ok)
    return sum(result) / len(result)
results = []

# Repeat the random sampling 100 times
for seed in range(100):
    # One random language per family
    sample = (info.groupby("family", group_keys=False).sample(1,
random_state=seed))
    # Glottocodes of selected languages
    codes = sample["Glottocode"]
    # Vowels from the selected languages
    sample_vowels = vowels[vowels["Glottocode"].isin(codes)].copy()

    # -- First universal --
    # Basic vowel symbols for each language
    base_sets = (sample_vowels.groupby("Glottocode")["base"].apply(set))
    # Proportion containing i + a + u together
    triangle = base_sets.apply(lambda x: {"i", "a", "u"}.issubset(x)).mean()

    # -- Second universal --
    nasal = sample_implication(sample_vowels, "nasal")
    long = sample_implication(sample_vowels, "long")

    # y -> i + u
    phoneme_sets = (sample_vowels.groupby("Glottocode")["Phoneme"].apply(set))
    # Languages containing y
    y_sets = phoneme_sets[phoneme_sets.apply(lambda x: "y" in x)]
    # Among languages with y, proportion also containing i and u
    y_result = y_sets.apply(lambda x: "i" in x and "u" in x).mean()
    results.append([triangle,nasal,long,y_result])

# One row = one random sampling
results = pd.DataFrame(results,columns=["i+a+u", "nasal", "long", "y"])
```

```
62
63  # Summary of the distributions
64  print(results.describe())
65  # 95% interval, less sensitive to extreme random samples than min/max
66  print(results.quantile([0.025, 0.975]))
```

```
       i+a+u     nasal      long         y
count 100.000000 100.000000 100.000000 100.000000
mean    0.834907   0.933957   0.762454   0.876893
std     0.011792   0.027992   0.028170   0.097758
min     0.806691   0.859649   0.689189   0.500000
25%     0.825279   0.911889   0.745757   0.833333
50%     0.836431   0.942308   0.765432   0.857143
75%     0.843866   0.946664   0.782191   1.000000
max     0.862454   1.000000   0.850746   1.000000
      i+a+u    nasal     long      y
0.025 0.812175 0.882904 0.700565 0.689286
0.975 0.858736 0.982615 0.809840 1.000000
```

Code block

```
1  results.describe()
2  results.quantile([0.025, 0.975])
```

	i+a+u	nasal	long	y
0.025	0.812175	0.882904	0.700565	0.689286
0.975	0.858736	0.982615	0.80984	1

I repeated the genealogically balanced sampling 100 times, each time randomly selecting one language per family and recomputing the proportions.

The mean proportion for the first universal ([i] + [a] + [u] or close equivalents) was 83.5%, with a 95% interval of about 81.2%–85.9%. For the second universal, the mean proportions were 93.4% for nasal → oral, 76.2% for long → non-long, and 87.7% for [y] → [i] + [u].

The first three results are relatively stable across the samples. But the [y] → [i] + [u] result varies much more, with a 95% interval from about 68.9% to 100%.

Overall, the main conclusions remain similar, but some results are more sensitive to which language is selected from each family. So it is better to report the mean proportions and their intervals rather than a single result from one random sample.

32. **In the light of these results, what exactly does a statement such as "X % of the world' s languages have [i]" mean?**

It means that X% of the languages in the sample have [i], under the choices used to build and analyse that sample. The percentage can change because some language families are more represented than others, and because different sampling choices give slightly different results. It is better to interpret X% as an estimate of a cross-linguistic tendency, not as a perfectly precise percentage for all languages.

5 Discovering Regularities in the Corpus

The idea now is to see whether it is possible to identify other regularities from the data contained in the corpus. To do this, we will ..... See `lab_language_comparison.pdf`

Using the PHOIBLE dataset:

33. **For each possible itemset of size 1 (single phonemes), compute its support (proportion of languages containing that phoneme).**

Code block

```
# presence: language × phoneme presence/absence matrix,  already created in Q25
# rows = languages, columns = phonemes
# True = the language has this phoneme, False = it does not
presence = pd.crosstab(df_selected["Glottocode"], df_selected["Phoneme"]) > 0

support_1 = (presence.mean().sort_values(ascending=False).reset_index())
support_1.columns = ["Phoneme", "Support"]
print(support_1.head(5))
```

	Phoneme	Support
0	m	0.965201
1	i	0.942766
2	k	0.907509
3	u	0.900183

```
4    j 0.900183
```

34. For all itemsets of size 2 (pairs of phonemes), compute their support in the same way.

Code block

```
# presence: language × phoneme matrix, already created in Q25 / Q33
X = presence.to_numpy(dtype=np.int32)
X = presence.to_numpy(dtype=np.int32)
phonemes = np.array(presence.columns)
# Number of languages containing each pair of phonemes
both = X.T @ X
# All possible phoneme pairs, without duplicates
i, j = np.triu_indices(len(phonemes), k=1)
support_2 = pd.DataFrame({
    "Phoneme1": phonemes[i],
    "Phoneme2": phonemes[j],
    "Support": both[i, j] / len(presence)
})
support_2 = support_2.sort_values("Support",
ascending=False).reset_index(drop=True)
print(support_2.head(5))
```

```
  Phoneme1 Phoneme2  Support
0        i        m 0.909799
1        i        u 0.892399
2        k        m 0.874542
3        j        m 0.873626
4        m        u 0.869048
```

35. Define as frequent itemsets those whose support exceeds the minimum support threshold chosen for this practical (e.g. 10% of all languages).

Code block

```
min_support = 0.10
frequent_1 = support_1[support_1["Support"] > min_support]
frequent_2 = support_2[support_2["Support"] > min_support]
```

Using a minimum support threshold of 10%, I found 77 frequent itemsets of size 1 and 1,124 frequent itemsets of size 2.

36. **Identify the frequent itemsets of size 2 and comment on any regularities you observe.**

Code block

```
print(frequent_2.sort_values("Support", ascending=False).head(10))
```

	Phoneme1	Phoneme2	Support
0	i	m	0.909799
1	i	u	0.892399
2	k	m	0.874542
3	j	m	0.873626
4	m	u	0.869048
5	i	k	0.863095
6	i	j	0.847985
7	a	m	0.846154
8	a	i	0.842949
9	k	p	0.838370

The most frequent phoneme pairs have very high support. For instance, [i, m] appears in about 91.0% of languages, and [i, u] in about 89.2%. Phonemes such as [m], [i], [u], [k] and [a] appear often. All of them are common phonemes without additional diacritics.

37. **For at least one of the frequent itemsets you identified, compute the confidence and lift for the association rule X → Y where X and Y are individual phonemes within that itemset.**

Code block

```
def association_rule(X: str, Y: str) -> dict:
    """
    X, Y: Two phonemes in the association rule X -> Y.
    return: Support of the pair, confidence and lift.
    """
    support_X = support_1.loc[support_1["Phoneme"] == X, "Support"].iloc[0]
    support_Y = support_1.loc[support_1["Phoneme"] == Y, "Support"].iloc[0]

    pair = support_2[
```

```
            ((support_2["Phoneme1"] == X) & (support_2["Phoneme2"] == Y)) |
            ((support_2["Phoneme1"] == Y) & (support_2["Phoneme2"] == X))
        ]
        support_XY = pair["Support"].iloc[0]

        # confidence = P(Y | X)
        confidence = support_XY / support_X
        # lift = P(Y | X) / P(Y)
        lift = confidence / support_Y

        return {"Rule": f"{X} -> {Y}","Support": support_XY,
            "Confidence": confidence,"Lift": lift}

    tests = [
        ("i", "m"),
        ("i", "u"),
        ("k", "m"),
        ("j", "m"),
        ("a", "i")
    ]
    tests_results = pd.DataFrame([
        association_rule(X, Y) for X, Y in tests])
    print(tests_results)
```

	Rule	Support	Confidence	Lift
0	i -> m	0.909799	0.965032	0.999824
1	i -> u	0.892399	0.946576	1.051537
2	k -> m	0.874542	0.963673	0.998416
3	j -> m	0.873626	0.970498	1.005488
4	a -> i	0.842949	0.957856	1.016007

I computed the support, confidence and lift for five frequent phoneme pairs.

38. Interpret your results: which phoneme combinations are most typical across languages? Do the confidence and lift values indicate stronger or weaker associations than expected under independence?

The most typical combinations are pairs of very common phonemes, such as [i, m], [i, u], [k, m], [j, m], and [a, i].

Their confidence values are generally very high, but most lift values are close to 1. This means that these phonemes often occur together mainly because they are already common

individually. The associations are mostly weak or only slightly stronger than expected under independence. The clearest positive association here is [i] → [u], with a lift of about 1.05. They often co-occur because both phonemes are already common. The presence of one phoneme usually adds little information about whether the other will occur.

To go beyond manually calculating support for pairs of phonemes, you can use an existing implementation of the Apriori algorithm, which efficiently discovers frequent itemsets of any size. One convenient implementation is provided in the Python library mlxtend (Machine Learning Extensions). Its apriori function takes a binary matrix (rows = languages, columns = presence/absence of phonemes) and returns all itemsets above a specified minimum support threshold. The same library also includes an association_rules function for computing confidence and lift automatically.

39. **Prepare your dataset as a binary matrix indicating, for each language, whether each phoneme is present (1) or absent (0).**

Code block

```
binary_matrix = presence.astype(int)
```

The binary matrix `presence` was already created in Question 25 using pandas.crosstab. It records, for each language and each phoneme, whether the phoneme is present (True) or absent (False). In the earlier questions, I used this matrix with NumPy and SciPy to compute pairwise statistics manually.

So for this question, the required binary matrix is therefore already available. I only convert the Boolean values to 1 and 0.

40. **Use the apriori function to identify frequent itemsets of size 2 and, if you wish, larger itemsets by adjusting the minimum support threshold.**

Code block

```
from mlxtend.frequent_patterns import apriori
frequent_itemsets = apriori(
    binary_matrix.astype(bool),   # mlxtend works better with Boolean data
    min_support=0.10,
    use_colnames=True,
    max_len=2
)
frequent_pairs = frequent_itemsets[frequent_itemsets["itemsets"].apply(len) ==
2]
```

```
print(frequent_pairs.sort_values("support", ascending=False).head(10))
```

```
     support itemsets
493 0.909799  (m, i)
509 0.892399  (u, i)
637 0.874542  (m, k)
584 0.873626  (m, j)
773 0.869048  (m, u)
488 0.863095  (i, k)
487 0.847985  (i, j)
97  0.846154  (m, a)
88  0.842949  (i, a)
644 0.838370  (p, k)
```

41. **Export the resulting itemsets together with their support values.**

Code block

```
frequent_pairs.to_csv("frequent_pairs.csv",index=False)
```

42. **Using the association_rules function, compute the confidence and lift for at least two association rules of your choice and comment on your findings.**

Code block

```
# 42 - confidence and lift using mlxtend
from mlxtend.frequent_patterns import association_rules
# Generate association rules automatically
rules =
association_rules(frequent_itemsets,metric="confidence",min_threshold=0)

# Convert antecedents and consequents to phoneme names
rules["X"] = rules["antecedents"].apply(lambda x: next(iter(x)))
rules["Y"] = rules["consequents"].apply(lambda x: next(iter(x)))

# Same five rules as in Q37
tests = [
    ("i", "m"),
    ("i", "u"),
```

```
    ("k", "m"),
    ("j", "m"),
    ("a", "i")
]

selected_rules = rules[
    rules.apply(lambda row: (row["X"], row["Y"]) in tests, axis=1)]

print(selected_rules[["X", "Y", "support", "confidence", "lift"]])
```

```
   X Y  support confidence    lift
23   a i 0.842949   0.957856 1.016007
833  i m 0.909799   0.965032 0.999824
865  i u 0.892399   0.946576 1.051537
1015 j m 0.873626   0.970498 1.005488
1121 k m 0.874542   0.963673 0.998416
```

I used mlxtend.association_rules() on the frequent itemsets found in Q40 and selected the same five rules as in Q37.

The results are very similar to the manual calculations. All five rules have high confidence, around 94.7%–97.0%, but their lift values are all close to 1.

This means that these phoneme pairs often occur together, but most of the association is explained by the fact that the phonemes themselves are already very common. The strongest positive association among these examples is [i] → [u], with a lift of about 1.05.

43. Run the analysis on vowels only, and then on the whole inventory. The itemsets computed over all phonemes are dominated by a handful of very frequent consonants: explain why, and say which of the two analyses answers the question we are interested in.

Code block

```
vowel_presence = pd.crosstab(vowels["Glottocode"],vowels["Phoneme"]) > 0
vowel_itemsets =
apriori(vowel_presence,min_support=0.10,use_colnames=True,max_len=2)
vowel_pairs = vowel_itemsets[vowel_itemsets["itemsets"].apply(len) == 2]

print("Whole inventory:")
print(frequent_pairs.sort_values("support", ascending=False).head(10))
print("\nVowels only:")
```

```
print(vowel_pairs.sort_values("support", ascending=False).head(10))
```

Whole inventory:

	support	itemsets
493	0.909799	(i, m)
509	0.892399	(i, u)
637	0.874542	(k, m)
584	0.873626	(j, m)
773	0.869048	(u, m)
488	0.863095	(i, k)
487	0.847985	(j, i)
97	0.846154	(a, m)
88	0.842949	(i, a)
644	0.838370	(k, p)

Vowels only:

	support	itemsets
92	0.892399	(i, u)
28	0.842949	(i, a)
34	0.803114	(u, a)
63	0.646062	(i, e)
89	0.629121	(o, i)
69	0.619048	(e, u)
25	0.601648	(e, a)
66	0.600275	(o, e)
114	0.598901	(o, u)
31	0.582875	(o, a)

In question 40, I used Apriori on the whole phoneme inventory, and here I repeated the same analysis with vowels only.

With the whole inventory, many top pairs contain consonants such as [m], [k], [j] and [p]. These consonants are already very frequent across languages, so they easily appear in many high-

support pairs.

With vowels only, the main pairs are [i, u], [i, a], [u, a], [i, e] and [o, i]. The support values are generally lower, and only three pairs are above 0.8. This shows that high co-occurrence is concentrated in a smaller number of vowel combinations.

For this lab, the vowels-only analysis is more useful because it removes the effect of very frequent consonants and makes the patterns inside vowel systems easier to see, which is our main focus.

44. **Vary the minimum support threshold (for instance 5 %, 10 %, 25 %) and report how the set of frequent itemsets changes. How would you choose a threshold, and what does this sensitivity tell you about the method?**

Code block

```
# 5%
itemsets_05 = apriori(
    vowel_presence,
    min_support=0.05,
    use_colnames=True,
    max_len=2
)
pairs_05 = itemsets_05[itemsets_05["itemsets"].apply(len) == 2]

# 25%
itemsets_25 = apriori(
    vowel_presence,
    min_support=0.25,
    use_colnames=True,
    max_len=2
)
pairs_25 = itemsets_25[itemsets_25["itemsets"].apply(len) == 2]

# Compare with the 10% result from Q43
print("5% threshold:", len(pairs_05), "frequent pairs")
print("10% threshold:", len(vowel_pairs), "frequent pairs")
print("25% threshold:", len(pairs_25), "frequent pairs")
```

> 5% threshold: 198 frequent pairs
>
> 10% threshold: 115 frequent pairs
>
> 25% threshold: 33 frequent pairs

Raising the minimum support threshold reduces the number of frequent pairs quite a lot: there are 198 pairs at 5%, 115 at 10%, and 33 at 25%. With a lower threshold, more patterns are kept, including less common ones. With a higher threshold, the analysis focuses only on the most frequent combinations, but some potentially interesting patterns may be excluded. The choice of threshold depends on how strict we want the analysis to be, and the results can change noticeably with this parameter.

45. **Find a rule whose confidence is high but whose lift is close to 1. Explain why confidence alone is misleading, and what lift adds.**

For [a] → [i], the confidence is about 95.8%, but the lift is only 1.016. So, most languages with [a] also have [i], which makes the rule look very strong at first.

But confidence alone is misleading, because it only tells us that [i] is very common when [a] is present. It does not consider that [i] is already very common in general.

Lift adds this baseline comparison. Since the lift is almost 1, [a] does not really make [i] much more likely to appear.

46. **Do the rules discovered by Apriori recover the universals you tested earlier? Do they suggest any regularity that was not part of your initial hypotheses — and how would you check whether such a regularity is real?**

Apriori does not directly recover the two universals tested earlier. The second universal is not really captured here, because this analysis only looks at exact phoneme co-occurrence and does not represent marked vowels and their corresponding simpler vowels.

For the first universal, the analysis is limited to itemsets of size 2, so it cannot test the complete [i, a, u] triangle. However, the three most frequent vowel pairs are [i, u], [i, a] and [u, a], all with support above 80%, showing that these vowels very often occur together in pairs.

The results also suggest another candidate regularity: pairs involving [e] or [o] together with [i], [a] or [u] still occur in about 58–65% of languages. This is a recurring pattern in the corpus, but support alone does not show that the association is strong. I would check its confidence and lift, and then test whether the pattern remains significant and stable in genealogically balanced samples.

6 Wrapping Up

47. **PHOIBLE contains the languages that someone has described and published, not a random sample of the world’ s languages. Which languages are likely to be missing, and in which direction could this bias the universals you have tested?**

PHOIBLE is not a random sample of the world’ s languages. Languages that are less studied, spoken by small communities, or have little published documentation are more likely to be missing.

This can affect the results if the missing languages are different from those already included in PHOIBLE. If they contain more exceptions to the universals, our results may overestimate how common these patterns are. If they follow the universals more often, the results may underestimate them.

So, we cannot know the direction of the bias without knowing more about the missing languages. The important point is not only how many languages are missing, but also what kinds of languages are missing.

48. Go back to the two universals stated in the introduction. For each, say what you now believe, what evidence supports it, and what would be needed to settle the question properly.

The first universal seems better described as a strong statistical tendency rather than an absolute universal. In the PHOIBLE sample, 79.8% of languages contain [i], [a] and [u] together, and this increases to 86.4% when close equivalents are included. The result also remains relatively stable after genealogical balancing. This gives fairly strong support to the general tendency, although there are still too many exceptions to call it absolute.

The second universal gives a less uniform picture. The nasal → oral implication is satisfied in about 88.4% of the relevant languages, the long → non-long implication in about 77.6%, and the [y] → [i]+[u] implication in about 89.8%. So the implicational universal is supported overall, but the strength of the pattern depends on the type of marked vowel.

A firmer conclusion would require broader and less biased language coverage, better control of family and area effects, and closer checking of the apparent exceptions. It would also help to define “close equivalent” and “corresponding simpler vowel” more consistently, since some counterexamples may come from transcription or source choices rather than from the vowel systems themselves.

49. More generally: what kind of observation would convince you that a proposed universal is wrong?

For an absolute universal, one well-established counterexample would be enough to reject it, provided that it is not caused by transcription or source problems. For a statistical universal, I would need stronger evidence: a substantial number of reliable counterexamples across different language families and areas, which remain after controlling for sampling and genealogical effects. I would also be doubtful if the result changed strongly under reasonable definitions or sampling choices.